Next-Generation AI Compiler Survey

Predicting the agentic compiler (~2027–28 and ~5 years): software-stack reshape and HW–SW codesign

Living survey export
Generated: 2026-08-03
Source repository: ai-compiler-survey
Primary narrative: docs/SURVEY.md · Roadmap: docs/ROADMAP.md · Stack: docs/STACK.md

> **North star.** Agents own semantic search, orchestration, and artifact synthesis. Compilers own lowering, legality, measurement, and fallback. Hardware codesign enters only through kernels, IR, tests, and profilers — not autonomous tape-out.

Survey narrative

Last updated: 2026-07-31
Companion digests: `../publications/`
Status: `../STATUS.md`
Conflicts: `CONFLICTS.md` · **Repos map:** `REPOS.md` · **Products:** `PRODUCTS.md`

0.1 North star

Primary goal: Predict the **next-generation agentic compiler** (architecture + process through ~2027–28 and ~5 years), including how it reshapes the **software stack** and **HW–SW codesign** — without drifting into general EDA.

Everything else (papers, GitHub/Gerrit, commercial SKUs, forums, ASIC bring-up studies) is **evidence** for that prediction—not a catalog for its own sake. When sources disagree, they go in `CONFLICTS.md` rather than being silently averaged.

Executive verdict. Compilation is shifting from **fixed pass pipelines** + **black-box autotuning** toward **hybrid LLM–compiler loops**. Empirically, the winning pattern is:

> **Agents own semantic search, orchestration, and artifact synthesis. Compilers own lowering, legality, measurement, and fallback.**

Agents reshape the **control plane** more than they replace the **data plane**. A fourth job — **accelerator bring-up / codesign feedback** on sim+silicon — is now Tier A evidence (TritorX, KernelEvolve), still centered on kernels/IR/oracles. See `ROADMAP.md`, `STACK.md`, §5, §6, §4.

---

1. What's the trend now? (Q1)

1.1 Two stacks converging

Stack	Question it answers	Maturity
Compilers for AI	How do we lower neural graphs to GPUs/TPUs efficiently?	Mature substrate (TVM/XLA/ MLIR/Triton); still fragmented
AI for compilers	How do we choose passes, write heuristics/kernels, use feedback?	Rapid 2023–2026 growth; hybrid systems dominate

"Next generation" is not "throw away LLVM/MLIR." It is **making those stacks agent-addressable**: structured summaries, bounded candidate spaces, tool APIs, and reward oracles.

1.2 Era timeline

Era	Label	What changed
2018–2022	DL compilers mature	TVM/Ansor, XLA, Glow; MLIR born from TF/XLA needs; cost-model autotune
2020–2023	ML-for-compiler RL	Autophase, CompilerGym, MLGO inlining/regalloc; neural policies hard to ship
2023–2024	LLM enters IR	Pass-list LLMs; Meta LLM Compiler foundation models; compiler feedback loops
2025–2026	Agentic hybrid era	Multi-agent tuners, verifier-RL, heuristic synthesis, Triton kernel agents, generative compilation, vendor CompileIQ / GEAK

1.3 Six active trends (detailed)

Trend A — Hybrid guidance, not LLM-as-compiler

Direct LLM IR rewrite is repeatedly fragile. mlirAgent reports frontier models scoring **below identity** on IR transforms. Systems that succeed constrain the LLM:

- **AgentCompile:** LLM emits advisory metadata only; templates + checks + benchmarks admit CUDA.
- **HintPilot:** LLM inserts compiler-validated pragmas/attributes, not arbitrary rewrites.
- **Meta LLM Compiler / Compiler-R1:** LLM proposes pass sequences; `opt` applies them.

Trend B — From RL gyms to LLM agents

CompilerGym exposed LLVM passes as OpenAI Gym environments (Autophase features, IR instruction-count rewards). That lowered the barrier for RL researchers but policies were often opaque and brittle across compiler versions.

2024–26 shifts toward:

1. **Foundation models** pretrained on IR/asm (Meta LLM Compiler: 546B tokens).
2. **Tool-using agents** trained with SFT+RL (Compiler-R1).
3. **Inference-only multi-agent tuners** that avoid heavy offline training (AutoPass).
4. **Program-synthesis of heuristics** that land as ordinary C++ (Magellan), recovering deployability that neural-in-the-compiler lacked.

Trend C — MLIR + Triton as default substrate

Practical GenAI path today:

```
PyTorch → Dynamo/Inductor → Triton → PTX/CUDA (or CUDA Tile IR)
```

Parallel portable path:

```
Framework → StableHLO → XLA or IREE → device backends
```

MLIR remains the shared engineering substrate even when product branding differs (OpenXLA, Triton internals, vendor AI compilers, CIRCT). Industry critique (Modular/Lattner) notes fragmentation and that open MLIR AI dialects have not always matched CUDA peak for LLM inference—hence Triton, CUTLASS, FlashAttention, and now CUDA Tile / CompileIQ.

Trend D — Kernel agents go industrial

Writing GPU kernels is the bottleneck for new model architectures and non-NVIDIA portability.

- **KernelBench (ICML 2025):** 250 PyTorch tasks; metric `fast_p` = correct **and** faster than baseline. Frontier models still often <20% one-shot; iterative refinement helps.
- **KernelBench-X:** refinement raises correctness but can **hurt** average speedup; many correct kernels still lose to eager; fusion tasks remain hard.
- **GEAK (AMD):** generator/evaluator/reflector/optimizer loop for Triton on Instinct; reported up to ~63% execution accuracy and ~2.59× speedup on suites.
- **Meta KernelLLM:** 8B model specialized for PyTorch→Triton; competitive Pass@k vs much larger general models on KernelBench-Triton.
- **AgentCompile:** compiler-bounded CUDA specialization for transformer inference graphs.

Trend E — Verification enters the loop

Correctness strategies, from weakest to strongest (local):

1. Compile + unit tests / LLM-generated tests (ACCLAIM).
2. Numerical checks vs reference path (AgentCompile).
3. Round-trip / recompile checks (LLM Compiler disassembly).
4. Formal semantic equivalence (Alive2 in LLM-VeriOpt rewards).

Formal coverage is still mostly **local peephole / IR**. Whole-program GPU races and FP nondeterminism lack equally strong oracles.

Trend F — Compilers broaden their object

- **Generative Compilation:** sealors complete partial Rust so the compiler can diagnose **during** LLM decoding—not only after file completion.
- **Compiler.next:** treat FMware (prompts, agents, free parameters) as a multi-objective search/compile problem for SE 3.0.
- **Anthropic Claude C Compiler:** agent teams *build a compiler* (~100kLoC Rust) capable of Linux kernel builds—adjacent but important: agents as compiler engineers, not only compile-time optimizers.
- **Magellan / AlphaEvolve:** agents rewrite the heuristic C++ inside production compilers (Google reports production inlining usage and XLA experiments).

1.4 Venue map

Community	What to watch
CGO / CC / ASPLOS / PLDI / MLSys	Systems + compilers
NeurIPS / ICML (+ C4ML)	Methods + KernelBench / Reasoning Compiler / Compiler-R1
ACL Findings	HintPilot-style SE/NLP crossover
LLVM Discourse (LLVM ♥ ML workshop)	MLGO, Magellan, agent PR review
Vendor blogs	NVIDIA CUDA Tile/CompileIQ, AMD GEAK, Meta KernelLLM/LLM Compiler, DeepMind AlphaEvolve, Modular

---

1b. Traditional AI compilation vs following trends

This section compares the **classic AI/DL compiler stack** (graph capture → fixed/autotuned lowering → kernels → runtime) with the **2024–2026 agentic / LLM-hybrid trends** summarized above. The point is not "old bad / new good," but where each side wins and what the hybrid should keep.

What "traditional" means here

Layer	Traditional approach	Representative systems
Front-end	Trace/export graphs; op fusion by rules	TorchDynamo/Inductor, TF/JAX → HLO/StableHLO
Mid-end	Dialect lowers + handwritten passes	MLIR, XLA, TVM Relay/TIR
Schedule / autotune	Cost models + evolutionary / template search	AutoTVM, Ansor, MetaSchedule
Heuristics	Expert C++ thresholds (inline, regalloc, …)	LLVM `-O3`, MLGO neural advisors (still in-tree)
Kernels	Libraries + expert CUDA/Triton/ CUTLASS	cuDNN, FlashAttention, hand Triton
Correctness	Compiler semantics + unit/golden tests	Deterministic passes; limited formal at scale
Serving	Separate runtime stack	vLLM, TensorRT-LLM, CUDA Graphs

Dimension-by-dimension comparison

Dimension	Traditional AI compilation — pros	Traditional — cons	Following trends (LLM/agent hybrid) — pros	Trends — cons / new risks
Correctness model	Decades of pass engineering; miscompile rates low for common paths	Misses intent-level opts; hard to prove GPU/FP whole-program	Gates (Alive2, tests, templates) can keep compiler as source of truth	Untamed LLM rewrite is unsafe; oracles still local
Search / adaptation	Autotune finds strong schedules given enough trials	Sample-inefficient; weak transfer across shapes/HW/ versions	Language priors + MCTS/agents cut samples; multi-level creativity	Token cost, nondeterminism, hard-to-replay traces
Heuristic maintenance	Human-readable, reviewable C++	High expert cost; stale vs new HW/ models	Magellan-style synthesis of deployable heuristics; MLGO learned advisors	Ownership, regression, supply-chain of agent code
Kernel specialization	Peak performance when experts invest	Does not scale to every new op/HW (esp. non-NVIDIA)	GEAK/KernelLLM/ AgentCompile automate explore–measure loops	Correct ≠ fast (KernelBench-X); fusion still hard
Production readiness	Default paths in PyTorch/XLA/LLVM/ CUDA	Fragmented stacks; peak often needs vendor libs	Vendor moves (CompileIQ, GEAK, Magellan prod inlining) show traction	Few agent loops are *default* compile flags yet
Portability	StableHLO / MLIR aim at framework↔compiler portability	Dialects and Triton/Tile/PTX still siloed	Agents can retarget kernels across AMD/ NVIDIA in principle	No standard agent IR/tool contract across stacks
Compile object	Graph → binary/ kernels	Does not cover prompts, agents, FMware knobs	Compiler.next / generative compilation broaden the object	Early vision; quality gates immature
Human role	Experts write passes/ kernels; long review cycles	Bottleneck on rare expertise	Agents draft; humans review/ orchestrate	Review capacity & security become the bottleneck
Cost model	Compile-time CPU + autotune GPU hours (known)	Autotune walls for huge spaces	Can reduce search trials	Adds LLM $ / latency; budgets poorly standardized
Explainability	Pass lists and schedules are inspectable	Why a heuristic fired can still be opaque	Natural-language rationales + traces possible	Rationales may not match true causal opts

Pros of staying traditional (when to *not* force agents)

1. **Stable, regressable defaults** — `-O3`, Inductor, XLA pipelines are battle-tested across millions of builds.
2. **Deterministic CI** — same IR in → same binary out (modulo known nondeterminism); agent loops break that unless carefully cached.
3. **Clear ownership** — a pass has a maintainer; an agent trajectory often does not.
4. **Peak library kernels** — FlashAttention/CUTLASS still dominate many hot paths without an LLM in the loop.
5. **Formal/tooling maturity** — Alive2, LLVM lit, FileCheck, and vendor validators assume classical artifacts.

Pros of following trends (when agents earn their keep)

1. **Per-program / per-model specialization** beyond what global heuristics allow (pass order, hints, CompileIQ knobs).
2. **Faster exploration** of kernel and schedule spaces with language priors (Reasoning Compiler, GEAK).
3. **Closing the intent gap** — algorithm-level rewrites compilers cannot invent (ACCLAIM), when tests gate them.
4. **Compiler engineering leverage** — synthesize heuristics/passes (Magellan) or review opt PRs with domain tools (LLVM Discourse agent review).
5. **New surfaces** — mid-decode diagnostics (Generative Compilation); FMware multi-objective compile (Compiler.next).

Synthesis: keep the substrate, change the control plane

```
Traditional strengths to preserve
  deterministic lowering · library kernels · CI regressability · formal local checks

Trend strengths to adopt
  advisory search · multi-agent orchestration · heuristic synthesis · profile/verifier feedback

Anti-pattern
  replace opt/Inductor with unconstrained LLM codegen and hope tests catch it
```

Recommendation: treat traditional AI compilers as the **data plane** and agent/LLM methods as an optional **control plane** with admit/fallback. Measure success by (a) production default adoption, (b) replayable traces, (c) oracle strength—not by microbench speedups alone.

---

2. How do agents help AI compilation? (Q2)

2.1 Mechanisms (why it works)

1. **Sample efficiency** — Language priors propose plausible schedules/passes instead of blind evolutionary samples (Reasoning Compiler vs MetaSchedule-style search).
2. **Correctness envelope** — Constraining interventions to hints, templates, verified IR, or pass lists avoids unconstrained codegen failures.

3. **Multi-level creativity** — Compilers miss purpose-level rewrites; LLMs can restructure algorithms when tests gate acceptance (ACCLAIM).
4. **Compiler engineering itself** — Agents write maintainable heuristics/passes (Magellan, mlirAgent) rather than shipping opaque neural nets inside `opt`.
5. **Feedback densification** — Profilers, compiler diagnostics, Alive2, and Fast Feedback turn sparse rewards into iterative refine loops.

2.2 Closed loop (canonical)

See `TAXONOMY.md`. Variants:

Variant	Twist
Generative Compilation	Feedback mid-decode via sealors
Reasoning Compiler	LLM proposals expand MCTS nodes
Magellan	Evolutionary mutate of C++ heuristics + Vizier autotune
ACCLAIM	Multi-level budgeted agents + test agent
GEAK	Reflexion-style generate–eval–reflect–optimize on GPU
CompileIQ	Evolutionary search over **compiler internal knobs** (not source)

2.3 What agents should *not* own

- Unchecked IR/assembly emission as production code without admit gates.
- Silent replacement of numerical kernels without golden or statistical checks.
- Orchestration without reliable tool-calling (ACCLAIM: open models fail on malformed tool calls before code quality).

---

3. Can agents reshape compilation processes? (Q3)

Short answer: Yes for the **control plane**; no (so far) for wholesale replacement of deterministic lowering.

These reshape claims are **evidence for** §5 Future prediction. The hard limits below—and the gaps in §4—are the **blockers** to that predicted future. Tiered repo/product evidence: `REPOS.md`, `PRODUCTS.md`.

3.1 Old → new process map

Legacy process	Agent-reshaped process	Evidence
Fixed `-O2/-O3/-Oz` pipelines	Per-program pass search with LLM/RL policies	Compiler-R1, AwareCompiler, AutoPass, LLM Compiler
Hand-tuned heuristics in C++	Agents synthesize/evolve heuristic code	Magellan (LLVM inlining/regalloc; XLA)
Black-box evolutionary autotune	Context-aware LLM proposals + structured search	Reasoning Compiler (TVM + MCTS)
Compile → fail → human fix	Compiler feedback during partial generation	Generative Compilation
Experts write Triton/CUDA	Agent generate–eval–reflect–optimize	GEAK, KernelBench agents, KernelLLM
Single IR level optimization	Multi-agent choose abstraction level	ACCLAIM
Compile source → binary	Compile intent / FMware knobs	Compiler.next (vision)
Generic NVCC heuristics	Per-kernel evolutionary compiler controls	NVIDIA CompileIQ
Humans write the compiler	Agent teams implement a C compiler	Anthropic CCC (engineering experiment)

3.2 Hard limits

1. **Direct IR transformation** by LLMs can underperform identity; search/heuristic synthesis wins more reliably (mlirAgent).
2. **KernelBench-X:** iterative refine can raise pass rates while average speedup falls; ~46% of correct kernels still slower than eager in their setting.
3. **Tool-calling quality** of open models can break multi-agent compilers before code skill does (ACCLAIM).
4. **Formal verifiers** cover local equivalences; they do not yet bless end-to-end GPU serving stacks.
5. **Cost & reproducibility** of agent compile loops (tokens, nondeterminism, hardware noise) remain open (Compiler.next call-to-action).

3.3 Practical architecture recommendation

Design agent-shaped compilers as:

```
bounded candidate spaces
+ structured IR / region summaries
+ advisory LLM
+ deterministic materialize
+ empirical and/or formal admit
+ fallback
```

Split **offline** compiler-engineering agents (Magellan-style heuristic synthesis) from **online** compile-time agents (HintPilot / AgentCompile / CompileIQ).

4. What's missing / under-covered? (Q4)

The gaps below are not a separate "wishlist"—they are the **blockers to the §5 predicted future** (agent-addressable data plane, three agent jobs, first-class artifacts, classical defaults until CI proves agents). Coverage is uneven. Each gap spells out **what exists**, **what is missing**, **why it blocks that future**, and **what "done" could look like**. Digests: `../publications/`. Evidence maps (Tier A/B/C): `REPOS.md`, `PRODUCTS.md`.

Gap map (priority snapshot)

#	Gap	Severity now	Who feels it first
4.1	End-to-end production evidence	High	Framework / compiler vendors
4.2	Correctness at scale	High	Anyone shipping kernels or IR rewrites
4.3	Cost & reproducibility of agent loops	High	CI / release engineering
4.4	Cross-stack interoperability	High	Multi-backend platforms
4.5	Hardware-native agent interfaces	Medium–High	Agent + compiler tool builders
4.6	FMware / agent-app compilation	Medium	LLM-app / SE 3.0 platforms
4.7	Training data for compilers	Medium–High	Foundation-model builders
4.8	Human-in-the-loop compiler engineering	Medium	LLVM/XLA maintainer orgs
4.9	Security / supply chain	Medium (rising)	Prod infra & open-source
4.10	Unified benchmarks	High	Research + industry comparison

4.1 End-to-end production evidence

What exists. Many papers report kernel-, pass-, or microbench-level wins: Compiler-R1 on IR instruction count; HintPilot on PolyBench; Reasoning Compiler on selected serving kernels; AgentCompile / GEAK on model or kernel suites; Magellan talks claim production inlining use; NVIDIA CompileIQ quotes ≤15% on already-hot GEMM/attention.

What is missing. Few agent or LLM methods are the **default** path in PyTorch Inductor, OpenXLA/XLA, or LLVM trunk for arbitrary user programs. Wins are often:

- opt-in flags or research forks;
- evaluated on curated kernels, not full training/serving stacks;

• hard to attribute (custom GEMV / CUDA Graphs / KV cache vs “LLM guidance”).

Why it blocks progress. Without default-path evidence, orgs cannot justify always-on agent compile cost or review burden. Survey claims risk overstating readiness.

Done looks like. Public A/B on major frameworks (e.g., Inductor default vs agent-specialized decode path) with regression suites, rollout flags, and multi-month stability—similar to how MLGO reported persistent QPS gains after deployment.

4.2 Correctness at scale

What exists.

Oracle	Strength	Weakness
Compiler applies passes (LLM Compiler, Compiler-R1)	Semantics of passes inherited	Cannot invent new transforms safely
Unit / LLM-generated tests (ACCLAIM)	Catches many functional bugs	Under-approximates; flaky tests
Numerical checks vs reference (AgentCompile)	Good for kernels with goldens	Tolerance games; training vs infer numerics
Alive2 / formal (LLM-VeriOpt)	Strong local IR equivalence	Peephole/local; not GPU concurrency
Round-trip disassembly checks	Partial trust signal	Exact-match rates still modest

What is missing. Oracles for:

- **Whole-program** semantic equivalence after multi-level agent rewrites;
- **Floating-point** contracts (reassoc, TF32/BF16, reduction order);
- **GPU races**, async copies, warp divergence, and memory model bugs;
- **Serving-level** behavioral equivalence (sampling, KV layout, speculative decode).

Why it blocks progress. KernelBench-X already shows correct kernels can be slow; the dual risk is “fast but subtly wrong.” Formal methods that stop at peephole leave the highest-value agent actions (fusion, schedule, algorithm rewrite) under-governed.

Done looks like. Layered admit policy: local formal where possible → differential testing on shape grids → statistical serving checks → staged rollout. Shared open oracles for Triton/CUDA Tile IR, not only LLVM IR.

4.3 Cost & reproducibility of agent compile loops

What exists. Fast Feedback (~10× iterate vs full IR in-loop); CompileIQ produces portable Advanced Controls Files; some papers fix seeds or report sample budgets; Compiler.next explicitly calls for reproducible intent compilation and shared traces.

What is missing.

- Standardized **cost models** (tokens + compile minutes + GPU-hours per % gain);
- **Deterministic replay** of agent decisions across model versions;
- **Cacheable compilation traces** keyed by (IR hash, HW, compiler version, agent policy);
- CI policies for flaky speedups (hardware noise, DVFS, contention).

Why it blocks progress. A compile that costs $N of LLM calls and cannot be reproduced is not a compiler feature—it is an experiment. Release engineering will reject it.

Done looks like. Trace format + content-addressed cache (pass list / hints / ACF / kernel artifact) checked into build systems; budget SLOs; golden replay tests when upgrading the agent model.

4.4 Cross-stack interoperability

What exists. StableHLO for framework↔compiler graphs; MLIR as a shared *idea*; Triton as a de-facto GPU DSL; vendor bridges (experimental Triton→Tile IR).

What is missing. A portable contract for **agent-visible** state:

Concept	Needed across stacks	Today
Region / fusion candidate	Common schema	Ad hoc per paper
Constraint set (shapes, aliasing, HW)	Shared vocabulary	Embedded in prompts
Action space (pass, hint, schedule, template)	Enumerated & versioned	Closed per system
Reward / admit result	Structured feedback	Free-text logs
Artifact identity	Hashable outputs	Often lost

Dialects, Triton, CUDA Tile IR, PTX, and LLVM IR remain siloed; an agent tuned on one stack rarely transfers.

Why it blocks progress. Every new agent re-implements glue. Multi-backend companies cannot share learnings.

Done looks like. An "agent compile interface" RFC (perhaps on StableHLO or MLIR) defining summaries, actions, and admit records—similar in spirit to how PJRT standardized runtime plugins.

4.5 Hardware-native agent interfaces

What exists. mlirAgent: structural IR fingerprinting, knowledge graphs, MCP tool suites; Compiler-R1 tool calls (`instrcount`, etc.); GEAK hardware feedback loops; CompileIQ search spaces over NVCC/PTXAS internals (NVIDIA/CompileIQ); **SCM-side tool APIs** in Archer (`verify`, `difftest`, `trans`, workflow) bound to a local `llvm-project` checkout; Gerrit AI Agent Provider APIs for in-UI chat (generic LLMs unless extended).

What is missing. Standards, not demos:

- Common fingerprint / provenance for pass effects;
- Vendor-neutral MCP (or equivalent) tool schemas for “compile / verify / bench”;
- Safe sandboxes for executing agent-proposed kernels at scale;
- First-class exposure of HW counters to agents without scraping profiler text.

Why it blocks progress. Agents without structured interfaces fall back to brittle prompt-and-pray over `opt` stdout.

Done looks like. LLVM/MLIR/Triton tool APIs that agents can call with typed I/O; fingerprinting as a supported analysis; conformance tests for tool servers.

4.6 FMware / agent-app compilation

What exists. Compiler.next vision: compile prompts, agent topologies, and free parameters under multi-objective quality gates; generative compilation couples compilers into coding agents; industry agent harnesses (Claude C compiler) stress test construction.

What is missing. Mature analogues of DL compilers for FMware:

- Stable IRs for prompt/tool graphs;
- Compilation that **fails closed** when quality thresholds miss;
- Interoperability between “prompt compilers” and classical model compilers;
- Shared traces for community learning (Compiler.next call-to-action #10).

Why it blocks progress. LLM applications still tune by hand and folklore while DL graphs enjoy decades of compiler investment.

Done looks like. Reproducible FMware compile pipelines with gold labels, cost/latency/quality Pareto fronts, and CI that blocks regressions—parallel to how model zoos ship compiled artifacts today.

4.7 Training data for compilers

What exists. Meta LLM Compiler: 546B tokens of LLVM-IR/assembly + compiler-emulation instruction data; KernelLLM: ~25k torch↔Triton pairs (KernelBook); Compiler-R1 reasoning dataset (~19.6k); CompilerGym workloads; MLGO training corpora (often internal).

What is missing. Large, open, **versioned** corpora for:

- MLIR dialects (linalg, scf, affine, vendor dialects);
- Triton / Tile IR / PTX paired with schedules and performance labels;
- StableHLO graphs with lowering outcomes;
- Negative data (failed compiles, miscompiles, slow kernels) for critics/verifiers.

Why it blocks progress. Without data, Selector/Generator models stay LLVM-centric or overfit KernelBench. Follow-ons to Meta LLM Compiler for modern AI IRs remain sparse.

Done looks like. Public “ImageNet for compilers” 2.0: multi-IR, multi-HW, with licenses cleared for commercial research—building on CompilerGym’s original ambition.

4.8 Human-in-the-loop compiler engineering

What exists. Magellan produces reviewable C++ heuristics; **Archer** (paper, GitHub) agentically reviews **LLVM GitHub PRs** with Alive2/LLUBI evidence gates; LLVM Discourse threads report similar agent PR review experience; **Gerrit** hosts general AI review plugins (ai-code-review, ReviewAI, GerritForge provider) used in large-org change workflows; Anthropic CCC emphasizes harnesses; Lattner commentary stresses tests as the real product. See `REPOS.md` for the SCM map.

What is missing. Process answers, not only models:

- Who **owns** agent-written passes six months later?
- How do code reviews differ when the author is an agent?
- When do humans override agent heuristics after HW refresh?
- How to mix agent draft + expert finalize without review collapse?

Why it blocks progress. Productivity gains reverse into maintenance debt if agent artifacts are unowned. Generic SWE agents underperform on opt review without domain tools—so HITL design is research-worthy.

Done looks like. Documented workflows: agent proposes → automated oracle gate → human review checklist → ownership assignment → regression corpus update. Metrics for review time vs bugs escaped.

4.9 Security / supply chain

What exists. Sparse public discussion: Anthropic author notes unease about unverified deployed code; classical compiler supply-chain concerns (trusting opt, binary provenance); almost no dedicated papers in this survey’s catalog on adversarial agent kernels.

What is missing.

- Threat models for **malicious or buggy agent kernels** (silent wrong answers, side channels, DoS via pathological schedules);
- Provenance signing for agent-generated heuristics/kernels;
- Sandbox policies for compile-time execution of untrusted proposals;
- SBOM-like records: which model, prompt, tools, and admit gates produced an artifact.

Why it blocks progress. As Magellan/GEAK/CompileIQ-style artifacts enter production binaries, they become part of the trusted computing base.

Done looks like. Security reviews required for agent-admitted artifacts; reproducible builds; allowlists for online agents; red-team suites for kernel agents.

4.10 Unified benchmarks

What exists (fragmented).

Suite	Measures	Blind spot
CompilerGym / Autophase-style	LLVM IR size / pass RL	Not GPU serving
PolyBench / HumanEval-CPP (+ HintPilot)	CPU runtime with hints	Not IR agents or kernels
KernelBench / KernelBench-X / TritonBench	GPU kernel correct+fast	Not full serving graphs
Paper-specific serving kernels	Attention/MoE/MLP slices	Not comparable across papers
Vendor internal suites	Production truth	Closed

What is missing. A **shared ladder**:

1. CPU IR opt (size + runtime),
2. Single kernels (CUDA/Triton/Tile),
3. Fused regions,
4. Full LLM serving graphs (prefill/decode, batch, long context),
5. Multi-objective (latency, throughput, memory, quality),

with fixed hardware profiles, reference docks, and leaderboards that separate **correctness**, **performance**, and **cost-to-compile**.

Why it blocks progress. Cross-paper "speedup" comparisons are currently nearly meaningless; the field cannot tell if Selector/Generator methods are improving the same problem.

Done looks like. A community benchmark consortium (LLVM ♥ ML + MLSys + vendor tracks) publishing versioned tasks and forbidding cherry-picked single-kernel headlines without the full ladder.

Cross-cutting research agenda (from the gaps)

Theme	Near-term work	Depends on gaps
Admit & fallback standards	Spec for hybrid pipelines	4.1, 4.2, 4.3
Agent compile IR / tools	RFC + MCP schemas	4.4, 4.5
Open multi-IR corpora	MLIR/Triton/Tile datasets	4.7, 4.10
Serving-level oracles	Diff tests + statistical checks	4.2, 4.10
Provenance & HITL	Ownership + signing workflows	4.8, 4.9
FMware compile MVP	Quality-gated prompt/agent search	4.6, 4.3

Follow-on questions for an org adopting this

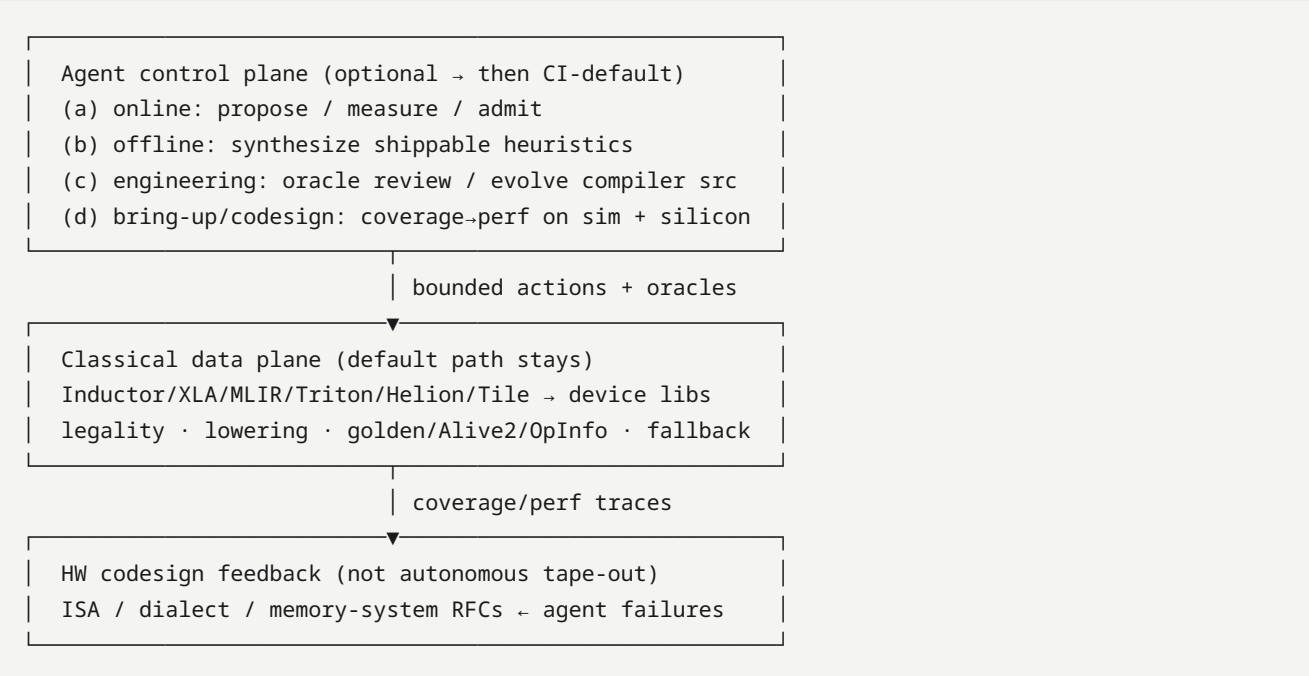
1. Online compile-time agents (HintPilot/AgentCompile/CompileIQ) vs offline heuristic synthesis (Magellan)—which matches your release model?
2. What is the correctness oracle—Alive2, golden kernels, or serving A/B—and who owns false negatives?
3. Which IR is the agent contract—LLVM IR, MLIR dialect, Triton, StableHLO, CUDA Tile IR?
4. How do you cache and regress agent compile traces across compiler *and* model upgrades?
5. What is the max $/build or tokens/build you will spend for a median X% win?
6. Who is the named maintainer of each agent-admitted artifact after merge?

---

5. Future prediction (what next-gen looks like)

Falsifiable sketch for **~2027–2028**, conditioned on conflicts in `CONFLICTS.md`.

5.1 Architecture



1. **Compiler becomes agent-addressable**, not agent-replaced — structured summaries, fingerprints, tool APIs, admit/fallback (mlirAgent: free IR rewrite loses to identity).
2. **Four agent jobs stick:** (a) online specialization, (b) offline heuristic/pass synthesis, (c) compiler engineering & review, (d) accelerator bring-up / codesign feedback.
3. **New first-class artifacts:** ACFs, evolved C++ heuristics, verified kernels, optimization memory, bring-up corpora, replayable traces.
4. **Defaults stay classical** until agents win on *distributions* in CI; hot kernels, size-critical apps, and *new ASICs* adopt first.
5. **Will not happen soon:** unconstrained LLM replaces `opt`/Inductor without oracles (**C6**); autonomous chip tape-out via compiler agents (**C10**).

5.2 How agents change the future (process)

Legacy	Agent-changed future	Confidence
Experts hand-write heuristics	Offline agents propose reviewable C++/MLGO features	High (Magellan/MLGO both shipping paths — conflict C1)
Autotune black boxes	Versioned search artifacts (ACF, traces) in VCS	Medium-high (CompileIQ design)
Scarce kernel experts	Multi-agent kernel loops with profiler oracles	Medium (vendor blogs strong; KernelBench-X ceilings — C2)
Human-only opt PR review	Compiler-oracle agents on PRs/ Changes	Medium (Archer; generic forge AI is not enough — C7)
Single DSL (CUDA) expertise	Multi-DSL agent skills (Triton/ Tile/CuTe/HIP)	Medium (TRT-LLM agents PR; GEAK v3 — C4)

5.3 What would falsify this prediction

- A major AI stack ships **default** lowering with no classical admit/fallback and sustained correctness.
- Magellan-class synthesis **and** MLGO neural advisors both disappear from production (neither path).
- Kernel agents plateau forever below eager on fusion-heavy suites with no industrial workaround (libraries-only forever).

5.4 Near-term signals conditioning the sketch

From Magellan LLVM Dev Meeting slides (digest), ACCLAIM (arXiv:2604.04238, digest), and HW-codesign agents:

Signal	Implication for §5	Watch
Magellan OSS via **OpenEvolve** + **OSS models**	Offline job (b) becomes reproducible outside Google	Public recipes / llvm patches (**C1**)
Magellan **XLA** auto-sharding / graph-rewrite green-field	Offline agents invent heuristics where human expertise is thin	End-to-end XLA pipeline eval (slides: WIP)
ACCLAIM multi-level compiler↔LLM cooperation (code)	Online job (a) is orchestration across levels + test admit, not IR replacement	Tool-calling quality; GPU/serving ports
TritorX coverage on MTIA + future-device sim	Job **(d) bring-up/codesign** enters the agentic compiler	Second-vendor repro (**C9**)
KernelEvolve hetero NVIDIA/ AMD/MTIA perf agents	Production multi-HW control plane	Public traces vs KernelBench-X (**C2**)
Ascend **compiler-grounded** Triton diagnosis	Non-CUDA NPUs need IR/pass escalation, not CUDA-pretrained guess	Hierarchy ablations
Helion + CompileIQ ACF path	DSL substrate agents specialize	**C4** vs Tile/CuTe

5.5 Five-year horizon & roadmap pointer

Detailed milestones: `ROADMAP.md` (Horizon A 2027–28, Horizon B ~2029–31).

Executive 5-year bet (still agentic-compiler-centric): classical data planes remain; agentic control planes become how orgs survive O(ops × devices × generations). HW codesign appears as **sim/silicon feedback into IR/ISA/dialects**, not autonomous tape-out (**C10**). New artifacts (ACFs, heuristics, memories, bring-up corpora) sit beside binaries in VCS.

5.6 Stack reshape pointer

Layer-by-layer SW + codesign map: `STACK.md`. Claim IDs: `CLAIMS.md`.

---

6. Conflicts (pointer)

Search results often disagree (vendor headlines vs docs, Magellan vs MLGO, free rewrite vs advisory-only, Triton vs Tile, online vs offline agents, coverage vs peak bring-up, compiler codesign vs autonomous EDA). **Do not collapse these into a false consensus.**

→ Full write-up: `CONFLICTS.md` (C1–C10).

Working stance used in §5: hybrid control/data plane; Magellan and MLGO as parallel bets; discount single-number speedups; constrain actions until oracles prove free rewrite; demote generic SCM AI as Tier C; codesign via coverage→perf agent ladder (**C9**), not autonomous chip design (**C10**).

---

7. How to read this repo

1. Skim **§0.1 North star**, **§5 Future prediction**, then `ROADMAP.md` / `STACK.md`.
2. Check `CLAIMS.md`; read `CONFLICTS.md` when two sources disagree.
3. Use `SYSTEMS.md` for concrete systems.
4. Use `REPOS.md` / `PRODUCTS.md` as **Tier A/B/C evidence for the prediction**, not forge/SKU catalogs.
5. Use `../publications/INDEX.md` (prefer ★ — ACCLAIM, Magellan, TritorX, KernelEvolve, Kernel*, CompileIQ, Archer).
6. Contribute via `WORKFLOW.md`; validate with `python3 scripts/validate_survey.py`.
7. Track progress in `../STATUS.md`.

One-page success check: (1) Predicted agentic compiler? → §5.1 / ROADMAP. (2) Four agent jobs including codesign bring-up? → §5.1 / STACK. (3) Evidence vs noise? → Tier A vs C.

Roadmap

North star: the **agentic compiler** — a hybrid control plane (LLM/agents + oracles) over a classical data plane (MLIR/LLVM/Inductor/Triton/Tile/vendor backends), including **HW–SW codesign** loops. Software catalogs and silicon bring-up are evidence for that target, not ends in themselves.

Companions: `SURVEY.md` §5 · `STACK.md` · `CLAIMS.md` · `CONFLICTS.md`

Horizon A — 2027–2028 (near)

Falsifiable sketch conditioned on C1–C10.

What ships

Capability	Predicted state	Leading evidence	Conflicts
Agent-addressable compilers	Tool APIs, structured IR summaries, admit/fallback become normal in LLVM/Inductor/vendor toolchains	ACCLAIM, HintPilot, AgentCompile, mlirAgent (negative free-rewrite)	C3, C6
Online specialization (job a)	Hot kernels/paths use agent or evolutionary search (ACF, hints, Triton/Helion refine) in CI for *some* products — not yet silent default for all builds	CompileIQ, GEAK, AutoKernel, Kernel Forge	C2, C5
Offline heuristic synthesis (job b)	Magellan-class C++ heuristic evolution *and* MLGO neural advisors both still live (parallel bets)	Magellan, EmitC-MLGO RFC	**C1**
Engineering agents (job c)	Compiler-oracle PR review (Alive2/`opt`) in serious LLVM/AI-compiler orgs; generic forge AI stays UX	Archer	**C7**
Bring-up / codesign agents (job d)	Coverage-first ATen/Triton backend generation on sim + silicon becomes standard for *new* ASICs	TritorX, KernelEvolve, Ascend hierarchical diagnosis	**C9**
DSL surface	Triton-family (Triton/Helion) remains primary agent training surface; Tile/CuTe/HIP/FlyDSL force multi-DSL skills	Helion, CompileIQ Helion path, TRT-LLM agents, KForge	**C4**

What does *not* ship by 2028

- Unconstrained LLM replaces `opt`/Inductor end-to-end without classical admit (**C6**).
- Single "one agent IR" for all vendors (**C4** unresolved).
- Kernel agents uniformly beat eager/libraries on fusion-heavy public ladders (**C2**).
- Agents design *silicon microarchitecture* autonomously (codesign is **feedback to humans/EDA**, not tape-out autopilot) — see Horizon B.

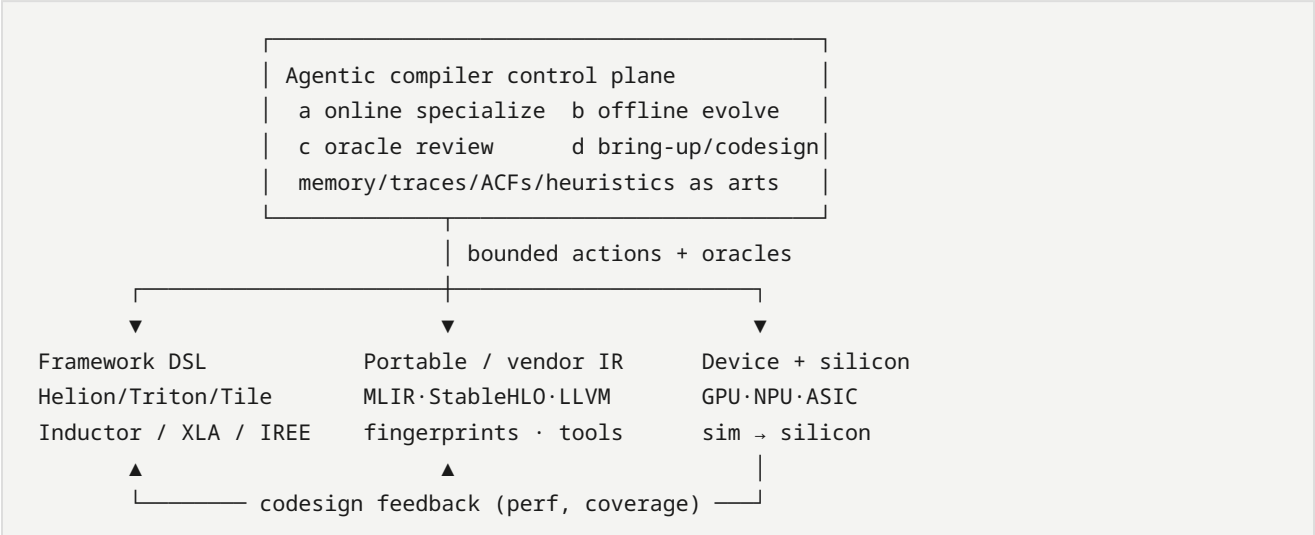
Near-term milestones (watch)

1. Public Magellan/OpenEvolve llvm patches **or** EmitC-MLGO default (**C1**).
2. CompileIQ/GEAK publish p50/p90 + pinned traces (**C2**).
3. TritorX-like bring-up reproduced outside Meta (second ASIC vendor) (**C9**).
4. PyTorch/LLVM release notes list agent/ACF jobs as supported workflows (**C5**).

Horizon B — ~2029–2031 (next ~5 years from 2026)

Still centered on the **agentic compiler** as the product; HW codesign is how that product eats the O(ops × devices × gens) matrix.

Architecture evolution



Predicted shifts

Theme	5-year outcome	Confidence
Default compile path	Classical lowering remains default; agentic specialize is **opt-in then CI-gated default for hot paths**	Medium-high
Artifact store	VCS grows first-class **ACFs, evolved heuristics, verified kernels, optimization memory, bring-up corpora**	High
Heterogeneous serving	Agentic multi-backend kernel/forge loops are how non-NVIDIA fleets stay viable (MTIA/AMD/Intel/NPU)	Medium-high (KernelEvolve, KForge, TritorX)
HW codesign loop	Pre-silicon: agents + sim generate coverage and compiler stress; post-silicon: agents map ISA/IR pain → RFC for next chip / dialect. Humans + EDA still own tape-out	Medium
Verification	Local formal (Alive2-class) + statistical serving oracles + OpInfo-scale suites compose; whole-program GPU/NPU formal still incomplete	Medium
Human role	Experts own oracles, ownership, security, ISA contracts; agents draft kernels/heuristics/backends	High
Will still not happen	Fully autonomous chip + compiler co-generation without human architectural intent; end-to-end LLM-as-`opt`	High

Codesign-specific roadmap (still agentic-compiler-centric)

Phase	Agentic compiler does	Hardware team gets
Pre-silicon	TritorX-style coverage on QEMU/sim; KernelEvolve-style search under draft ISA docs	Early “can we run NanoGPT/DLRM ops?” signal; IR/dialect bugs
Bring-up	Coverage agents → perf agents ladder	Weeks→hours for ATen/Triton backend skeleton
Steady-state	Online specialize + memory (KernelBlaster) across gens	Portability when L2/TMA/SRAM rules change
Next tape-out	Aggregated failure modes from agent traces (illegal ops, alignment, missing atomics)	Prioritized ISA / memory-system / compiler-pass requests

Non-goal: surveying EDA/RTL LLM tools unless they close the loop into **compiler IR, kernels, or admit oracles**.

Success metrics for this roadmap

1. Can a new ASIC expose an agent-addressable compile/test API and reach >80% ATen coverage via agents within a release cycle? (TritorX bar)
2. Do agentic specialize jobs show **distributional** wins in CI (p50), not only best kernels? (C2)
3. Are Magellan-class and MLGO-class paths both still productive or has one settled? (C1)
4. Does the stack treat traces/ACFs/heuristics as reviewable artifacts with owners? (§4.8–4.9)

Update when CONFLICTS settle or new Tier A codesign evidence lands.

Software stack & HW codesign reshape

Focus: not "AI software in general," but how an **agentic compiler** changes layers from framework UX down to silicon feedback.

Companions: `ROADMAP.md` · `SURVEY.md` §5 · `TAXONOMY.md` · `CONFLICTS.md`

Layer map (today → agentic)

Layer	Classical role	Reshape by agentic compiler	Evidence
1. Model / framework	`torch.compile`, JAX/TF export	Agents consume graphs/ regions; Amdahl-rank hot ops; write kernels back into eager/compile path	AutoKernel, Kernel Forge, AgentCompile
2. Kernel DSL	CUDA / Triton / Helion / Tile / CuTe / HIP	DSL becomes **agent training + search surface**; Helion raises abstraction; multi-DSL skills required	Helion, GEAK, CompileIQ, KForge, TRT-LLM agents PR
3. Portable IR	StableHLO, MLIR dialects	Must expose fingerprints, tool APIs, legality; free rewrite fails	mlirAgent, StableHLO, MLIR
4. Compiler mid/back	LLVM/XLA/Inductor/ NVCC passes	Offline agents evolve heuristics; online agents pick passes/hints/ACFs; MLGO advisors persist	Magellan, MLGO, ACCLAIM, HintPilot, CompileIQ
5. Oracles & profilers	Unit tests, Alive2, NCU	Become **admit gates + reward**; federated profilers (MPP) required for hetero HW	Archer, LLM-VeriOpt, KernelEvolve, Ascend diagnosis
6. Artifacts / VCS	Binaries, schedules	**ACFs, evolved C++, verified kernels, optimization memory, bring-up corpora**	CompileIQ, Magellan, KernelBlaster, TritorX
7. Serving runtime	vLLM, TRT-LLM, custom ads stacks	Agent loops specialize serving kernels; must not break graph-level opts	GEAK, KForge vs TRT-LLM, KernelEvolve
8. Silicon / sim	Manual bring-up, ISA docs	Agents generate backends on **sim + silicon**; traces inform next ISA/IR (codesign)	TritorX, KernelEvolve, Ascend NPU paper

Four agent jobs on the stack

```
(a) Online specialize     → layers 1-2-4-5-7   (CompileIQ, GEAK, AutoKernel, ACCLAIM)
(b) Offline evolve        → layer 4 (+ artifacts) (Magellan / AlphaEvolve)
(c) Oracle engineering    → layers 4-5-6         (Archer, CCC-adjacent)
(d) Bring-up / codesign   → layers 2-5-8         (TritorX, KernelEvolve, Ascend diagnosis,
KForge)
```

Job **(d)** is the HW-codesign extension: still an **agentic compiler/toolchain** problem (kernels, dialects, tests), not general chip LLM design.

Stack reshape theses (claim IDs)

ID	Thesis	Status
S1	Control plane becomes agentic; data plane stays classical	Supported — see CLAIMS A1
S2	Portability shifts from "write once IR" to "agent + oracle per backend" while IR remains necessary substrate	Contested — C4, C8
S3	New first-class artifacts (ACF/ heuristics/memory/traces) change CI and code review	Supported — A3
S4	Custom ASIC competitiveness increasingly depends on agentic bring-up latency	Supported (industrial) — TritorX/ KernelEvolve; watch second-vendor repro — C9
S5	Profilers and compiler internals move from human IDE tools to **agent APIs**	Watch — KernelEvolve MPP, Ascend hierarchy

What *not* to confuse with stack reshape

Lookalike	Why it is weaker for *this* survey
Generic coding agents on app repos	No compile oracles → Tier C
Pure EDA/RTL LLM without kernel/IR loop	Out of scope unless tied to compiler admit
Vendor SKU lists without agent/oracle APIs	Tier B baselines only

Reading order

1. `ROADMAP.md` horizons A/B
2. This file's layer table

3. `CLAIMS.md` A/*S*/P*
4. Digests ★ in INDEX under *HW codesign* and *GPU kernels*

Claims map

Living map from falsifiable claims to digests. Update when evidence moves status.

Status: **Supported** · **Contested** · **Watch** · **Falsified**

Architecture (agentic compiler)

ID	Claim	Status	Best evidence	Conflicts
A1	Agents own search/ orchestration/ synthesis; compilers own lowering, legality, measure, fallback	Supported	ACCLAIM, AgentCompile, HintPilot, mlirAgent (negative)	C3, C6
A2	Four agent jobs stick: (a) online, (b) offline heuristics, (c) engineering/ review, (d) bring-up/codesign	Supported	(a) CompileIQ/ GEAK/AutoKernel; (b) Magellan; (c) Archer; (d) TritorX/ KernelEvolve	C5, C9
A3	ACFs, evolved heuristics, verified kernels, optimization memory, bring-up corpora become first-class artifacts	Supported	CompileIQ, Magellan, KernelBlaster, TritorX	—
A4	Defaults stay classical until agents win on *distributions* in CI	Contested	Vendor blogs vs CompileIQ 2–3% docs, KernelBench-X	C2
A5	Unconstrained LLM will not replace `opt`/ Inductor soon	Supported	mlirAgent; hybrid Tier A dominance	C3, C6

Process & stack

ID	Claim	Status	Best evidence	Conflicts
P1	Offline heuristic synthesis and MLGO neural advisors remain parallel bets through 2028	Contested	Magellan vs EmitC-MLGO	**C1**
P2	Multi-DSL / multi-vendor agent skills become normal	Watch	KForge, GEAK v3, TRT-LLM agents, Helion+CompileIQ	**C4**
P3	Compiler-oracle review beats generic forge AI for opt PRs	Supported (direction)	Archer; Tier C demoted	**C7**
S1	Stack reshape is control-plane agentic over classical data plane	Supported	STACK.md · A1	C6
S4	Custom ASIC TTM increasingly gated by agentic bring-up	Supported (industrial)	TritorX, KernelEvolve	**C9**
S5	Profilers/compiler internals become agent APIs	Watch	KernelEvolve MPP, Ascend hierarchical diagnosis	C3

Codesign (still agentic-compiler-centric)

ID	Claim	Status	Best evidence	Conflicts
H1	Pre-silicon sim + agents provide compiler/ISA feedback before tape-out	Supported (early)	TritorX QEMU future devices	C9, C10
H2	Coverage-first agents then perf agents is the bring-up ladder	Supported	TritorX → KernelEvolve	**C9**
H3	Agents will not autonomously tape out chips by ~2031; they stress compilers/ISAs	Supported (prediction)	Scope of TritorX/ KernelEvolve (kernels/ toolchains)	**C10**

Settlement watch

Signal	Moves	Digests
Public Magellan llvm + OpenEvolve recipes	C1	magellan, openevolve
EmitC-MLGO default on Android/ Chrome	C1	mlgo-emitc-rfc
p50/p90 public ACF/kernel traces	C2	compileiq-*, kernelbench-x
Second non-Meta ASIC reproduces TritorX-class coverage	C9	tritorx
Agent IR contract where free rewrite beats advisors	C3	acclaim vs hintpilot/agentcompile

Conflicts register

This page records **disagreements across papers, vendor blogs, OSS repos, and forums** that matter for predicting the next-generation AI compiler and how agents change that future. We do **not** force a premature resolution; each conflict states both sides, why it matters for the prediction, and what would settle it.

Companion: `SURVEY.md` § Future · `PRODUCTS.md` · `REPOS.md`

How to read a conflict row

Field	Meaning
Claim A / Claim B	Competing readings from primary sources
Why it matters	How the winner changes the next-gen compiler sketch
Settlement signal	What evidence would decide it

C1 — Evolve shippable C++ heuristics vs embed neural advisors (Magellan vs MLGO)

Side	Sources	Position
A — Synthesize readable heuristics	Magellan paper/slides; AlphaEvolve lineage; OpenEvolve	Agents rewrite **EVOLVE-BLOCK** C++ inside LLVM/XLA; ship like human passes; Magellan claims inlining beats decades of manual work; slides hope to leapfrog NN policies
B — Keep/improve neural MLGO	LLVM Discourse EmitC RFC; IR2Vec+MLGO RFC; ongoing MLGO meetings (2026)	Production Chrome/Android/ Fuchsia still invest in **in-tree NN advisors**; EmitC/TOSA path removes TF build deps so neural policies stay deployable

Why it matters. If A wins, the next-gen control plane is an **offline evolutionary coding agent** whose output is ordinary compiler source. If B wins, the data plane keeps **learned policies** as first-class runtime advisors, and agents mainly help *train/feature* those NNs.

Settlement signal. Public Magellan heuristics land in llvm-project *and* displace MLGO on the same size/perf apps; or EmitC-MLGO becomes the default path for Android/Chrome while Magellan stays Google-internal.

C2 — Vendor “production agent” wins vs sober benchmark ceilings

Side	Sources	Position
A — Strong commercial wins	NVIDIA CompileIQ blog (Meta up to ~15% on TritonBench/Helion); AMD GEAK v3 blogs (repo-level HIP/Triton/FlyDSL); AlphaEvolve Cloud GA	Agent/autotune control planes already deliver meaningful production speedups
B — Hard ceilings & regressions	CompileIQ docs (often **2–3%** on highly optimized kernels); KernelBench / KernelBench-X (many correct kernels slower than eager; refine↑correctness can ↓avg speedup; fusion hard)	Headline speedups are workload-selected; iterative agents can chase correctness at the cost of performance

Why it matters. Prediction of “agents become default compile” needs median/CI wins, not only cherry-picked kernels. Overclaiming delays investment in oracles and traces (§4.2–4.3).

Settlement signal. Reproducible public ACF/kernel agent traces with fixed compiler versions, reporting **distribution** (p50/p90) not only best case; KernelBench-X-style fusion suites remain unsolved or get solved.

C3 — LLMs rewrite IR/code freely vs must stay advisory

Side	Sources	Position
A — Multi-level LLM rewrite works with tests	ACCLAIM (compiler–LLM cooperation); GEAK generate–eval–reflect; KernelAgent	Guiding agents interleave LLM rewrites with compiler tools; tests/profiles admit candidates; speedups reported
B — Direct IR transform fails	mlirAgent (frontier models **below identity** on IR transforms); HintPilot/AgentCompile design (hints/templates only)	Unconstrained IR rewrite is unsafe/weak; successful systems **constrain** the action space

Why it matters. Next-gen architecture either exposes a **wide rewrite API** (with strong oracles) or a **narrow advisory API** (hints, knob ACFs, heuristic blocks). These are different products.

Settlement signal. Shared agent IR contract + oracle suite where free rewrite consistently beats constrained advisors on correctness×perf; or industry standardizes on advisory-only admit gates.

C4 — Kernel DSL future: Triton vs CUDA Tile (and friends)

Side	Sources	Position
A — Triton remains the agent surface	Inductor default path; KernelBench; KernelLLM; GEAK Triton path; awesome-LLM-driven-kernel-generation catalog	Ecosystem, benchmarks, and agents already converge on Triton
B — Tile / CuTe / HIP / FlyDSL fragment the surface	NVIDIA CUDA Tile + CompileIQ; TRT-LLM Claude agents for CuTe/TileIR/Triton/CUDA; GEAK multi-language (HIP, FlyDSL, TileLang)	Vendors push hardware-native tile IRs; agents must become multi-DSL or lose peak

Why it matters. Training data, tool APIs, and "one agent IR" bets succeed or fail with this choice (§4.4, §4.7).

Settlement signal. One DSL becomes the dominant *agent training* corpus; or a portable tile IR wins; or multi-DSL agents (TRT-LLM skills pattern) become the norm.

C5 — Online compile-time agents vs offline compiler-engineering agents

Side	Sources	Position
A — Online (in the compile/serve loop)	CompileIQ ACFs; HintPilot; AgentCompile; GEAK on serving stacks; AlphaEvolve Cloud for algo search	Users pay tokens/GPU at optimize time; artifacts are configs/kernels per workload
B — Offline (change the compiler once)	Magellan; MLGO training; Archer PR review; Anthropic Claude C Compiler	Agents change **source of the compiler** or review PRs; users get classical `-O3`/`opt` afterward

Why it matters. These are two different "agent futures." A hybrid org may need both, but roadmaps and cost models differ.

Settlement signal. Which path shows up as the *default* flag or CI job in PyTorch/LLVM/CUDA release notes over 12–24 months.

C6 — Agents replace compilers vs agents are the control plane

Side	Sources	Position
A — Agents can build/replace large compiler surfaces	Anthropic CCC (~100kLoC Rust compiler); some HN/forum optimism	Agent teams author compilers; classical eng bottleneck shrinks
B — Hybrid control/data plane is the durable pattern	New Compiler Stack survey; mlirAgent limits; ACCLAIM cooperation framing; vendor stacks still ship TRT-LLM/ Inductor/XLA	Data plane (lowering, legality, measure) stays classical; agents search/synthesize/advise

Why it matters. Our survey's executive verdict bets on **B**. A would rewrite goals toward "agent-authored compilers" as the primary object.

Settlement signal. A production AI stack whose *default* lowering path is agent-generated without a classical admit/fallback compiler underneath—not a research demo.

C7 — Generic SCM AI review vs compiler-oracle review

Side	Sources	Position
A — Forge AI is enough	Gerrit ai-code-review / ReviewAI / native AI chat; generic GitHub PR bots	Put LLM on the diff; scale human review
B — Compiler-specialized tools required	Archer (Alive2/LLUBI/`opt`); LLVM Discourse agent-PR experience	Miscompiles need domain oracles; generic review is HITL UX only

Why it matters for *this* survey. Generic Gerrit plugins are **weak evidence** for next-gen *compilers*; Archer-class tools are strong. Cataloguing forge plugins without oracles misaligns with the prediction goal.

Settlement signal. A Gerrit/GitHub bot that blocks merge on failed Alive2/KernelBench-class checks becomes default in llvm-project or a major AI compiler.

C8 — "AI compiler" means DL graph compilers vs LLM-for-LLVM

Side	Sources	Position
A — Compilers for AI models	TVM/XLA/Inductor/TRT-LLM/ OpenVINO product docs	Next-gen = better graph→device stacks (Tile, StableHLO, Neuron NKI)
B — AI for compilers	Magellan, LLM Compiler, Compiler-R1, Archer	Next-gen = agents inside LLVM/ XLA/kernel eng

Why it matters. Commercial catalogs over-weight A; research catalogs over-weight B. Prediction must keep **both stacks converging**, with agents as the cross-cut—not pick one catalog.

Settlement signal. (Already partially here.) Products that expose agent APIs *on* DL compilers (CompileIQ, GEAK, Magellan→XLA) are the convergence proofs.

C9 — Coverage-first bring-up agents vs peak-performance kernel agents

Side	Sources	Position
A — Coverage unlocks the device	TritorX (481 ATen ops, OpInfo, MTIA sim+silicon); KForge on Intel Arc	New ASICs need *any correct* backend before peak kernels; agents should maximize operator coverage first
B — Perf agents are the product	KernelEvolve, GEAK, AutoKernel, KernelBench `fast_p`	Without speedups vs eager/ libraries, agentic compile does not pay TCO; coverage-only backends still lose to NVIDIA stacks

Why it matters. The agentic-compiler roadmap needs a **ladder** (coverage → perf) vs a single objective. Codesign programs that only optimize HotGEMM will strand models; programs that only chase OpInfo will never win serving.

Settlement signal. Public playbooks that sequence TritorX-class coverage then KernelEvolve-class perf on the same ASIC, with serving metrics — or one objective dominates release criteria industry-wide.

C10 — Agentic compiler codesign feedback vs autonomous chip design

Side	Sources	Position
A — Agents co-design silicon	Broad “AI for chip design” narratives; optimism from sim bring-up	LLMs will propose ISA/microarch with compilers in the loop end-to-end
B — Agents stress toolchains; humans/EDA own tape-out	TritorX/KernelEvolve actual scope (kernels, dialects, tests, profilers); this survey’s ROADMAP	Agentic *compilers* shorten SW TTM and file ISA/IR pain reports; autonomous tape-out is a different field

Why it matters. Keeps survey focused on the **target agentic compiler**. HW is in scope only when it closes the loop through kernels/IR/oracles.

Settlement signal. A production chip whose microarchitecture was primarily agent-proposed *and* validated via agentic compile oracles — not merely agent-written RTL fragments without compiler admit.

Working stance for this survey (until settlement)

1. Prefer **hybrid control/data plane** (C6-B) as the prediction baseline.
2. Treat Magellan-style **heuristic synthesis** and MLGO **neural advisors** as **parallel production bets** (C1 unresolved).
3. Discount single-number vendor speedups without distribution/oracle context (C2).
4. Assume **constrained actions** + **strong oracles** until free rewrite proves itself (C3).
5. Demote generic SCM AI plugins to Tier C evidence (C7).
6. Keep DL-compiler products as **Tier B baselines**, not as the definition of next-gen (C8).
7. Codesign via **coverage→perf agent ladder** on sim+silicon (C9); do **not** expand into autonomous EDA (C10-B).

Update this file when a conflict gains a decisive public settlement.

Systems comparison

Snapshot of representative systems. Numbers are **as reported by authors**; cross-benchmark comparison is not apples-to-apples. Prefer mechanisms over headline speedups when choosing architecture.

System	Layer	Agent shape	Correctness gate	Headline	Venue / source
Meta LLM Compiler	LLVM IR / asm	Foundation model	Compiler applies passes	~77% of autotune size potential	CC 2025
Compiler-R1	LLVM pass order	Single agent + tools + RL	opt + IR instr count	~8.5% IR size vs -Oz avg	NeurIPS 2025
LLM-VeriOpt	LLVM IR peephole	Trained small model	Alive2 equivalence	~90% verifiably correct	CGO 2026
Magellan	Heuristics (C++)	Coding agent + AlphaEvolve	Compile + macro-benches	Beats decades of inlining heuristics	C4ML@CGO'26 / LLVM DevMtg
AwareCompiler	Pass sequences	Multi-turn agent–env	Compile/run rewards	Knowledge bridges features↔passes	arXiv 2025
AutoPass	Pass / flags	Multi-agent, inference-only	Compile + profile evidence	Practical budgets, no offline RL	arXiv 2026
HintPilot	Source pragmas	Iterative RAG refine	Compiler validates hints + tests	Up to 6.88× geo-mean vs -Ofast	ACL Findings 2026
ACCLAIM	C / asm multi-level	Guide + level + test agents	LLM tests + compile	Up to 1.25× over compilers alone	arXiv 2026
Reasoning Compiler	TVM schedules	LLM proposals + MCTS	TVM compile/ measure	Sample-efficient vs MetaSchedule	NeurIPS 2025
AgentCompile	CUDA inference	Advisor in bounded search	Template CUDA + checks + bench	~4–5.7× vs eager (small LLMs)	arXiv 2026
GEAK	Triton / AMD GPU	Gen/eval/ reflect/optim	Unit tests + timing	Up to 63% exec acc; ~2.59×	AMD / arXiv 2025
KernelLLM	PyTorch→Triton	Specialized 8B model	KernelBench-Triton	Strong Pass@k vs larger general LLMs	Meta HF 2025
mlirAgent	MLIR / LLVM	Tool-using agents (MCP)	Pass tracking + benches	LLMs weak at direct IR rewrite	UCB BAR
Generative Compilation	Rust frontend	In-decoding feedback	Sealor + rustc	Compiler active during generation	arXiv 2026
Compiler.next	FMware / intent	SE 3.0 vision	Multi-objective quality gates	Compile prompts/agents/ params	arXiv 2025
NVIDIA CompileIQ	NVCC/PTXAS knobs	Evolutionary search	Measure on real kernels	≤15% on hot Triton/CUTLASS kernels	CUDA 13.3 blogs

System	Layer	Agent shape	Correctness gate	Headline	Venue / source
MLGO	LLVM heuristics	RL policies in-tree	Compiler semantics	Prod inlining/ regalloc	Google / LLVM
KernelBench	Eval harness	N/A (benchmark)	Correct + fast_p	Frontier <20% one-shot typical	ICML 2025
TritorX	PyTorch ATen / Triton-MTIA	FSM agent bring-up	OpInfo + model ops	481 ops; ~84% pass; sim+silicon	MLSys 2026
KernelEvolve	Triton (+TLX) multi-HW	Graph search + HW RAG	TritonBench + federated profilers	Hetero NVIDIA/ AMD/MTIA prod	Meta 2025/26
KForge	Multi-DSL / multi-vendor	Gen ↔ perf-analysis agents	Compile + correct + profile	+2.12% vs TRT-LLM; 5.13× on Arc L2	arXiv 2026
AutoKernel	Triton/CUDA on PyTorch models	Keep/revert agent loop	5-stage harness	Beats eager & torch.compile on hot ops	arXiv 2026
Ascend hierarchical diagnosis	Triton-NPU	Escalating compiler-grounded agents	Profile → IR → compiler source	4.35× geo-mean on 37 ops	arXiv 2026
Helion	PyTorch→Triton DSL	Autotune (not LLM)	Config search	Geomean > compile/Triton on reported suites	PyTorch 2025

Online vs offline agents

Mode	When it runs	Typical artifact	Examples
Online	At compile / specialize time for a program or model	Pass list, hints, kernel choice, ACF knobs	HintPilot, AgentCompile, CompileIQ, Compiler-R1 inference
Offline	Compiler engineering / model training	C++ heuristics, foundation weights, datasets	Magellan, Meta LLM Compiler training, MLGO training

Related engineering experiments (adjacent)

Work	Why it matters to this survey
Anthropic Claude C Compiler	Agents as *compiler writers*; harness + tests dominate
LLVM agent PR review Discourse thread	Compiler-specific tools >> generic SWE agents for opt review

Taxonomy

Two meanings of "AI compiler"

Sense	Meaning	Examples
Compilers for AI	Systems that lower neural graphs to accelerators	TVM, XLA/OpenXLA, MLIR dialects, TorchInductor→Triton, IREE, TensorRT
AI for compilers	LLMs/agents that choose passes, rewrite IR, write heuristics/ kernels	Meta LLM Compiler, Compiler-R1, Magellan, GEAK, HintPilot

This survey treats **next-gen** as their **merger**: agents on the control plane, compilers on the data plane.

LLM role taxonomy (New Compiler Stack, 2026)

From *The New Compiler Stack: A Survey on the Synergy of LLMs and Compilers* (arXiv:2601.02045):

1. **Selector** — choose among predefined compiler actions or candidates (pass lists, schedule moves, CUDA template families).
2. **Translator** — rewrite source / IR / assembly (highest correctness risk unless gated).
3. **Generator** — synthesize new compiler artifacts (heuristics in C++, kernels, tools, datasets).

Most strong 2025–26 systems are **Selectors or Generators wrapped in hybrid validation**, not free-form Translators alone.

Agent roles in the compile loop

Role	Job	Examples
Advisor / Selector	Rank candidates, label regions, suggest passes	AgentCompile, Meta LLM Compiler, AutoPass
Translator / rewriter	Source/IR/asm rewrite or hint insertion	ACCLAIM, HintPilot, LLM-VeriOpt
Artifact Generator	Heuristics, kernels, MCP tools	Magellan, GEAK, mlirAgent
Orchestrator	Budget, IR level, stop conditions	ACCLAIM guide agent, GEAK directors
Tester / critic	Tests, Alive2, profiles, refine prompts	ACCLAIM test agent, Generative Compilation
Search partner	Propose nodes for MCTS / evolution	Reasoning Compiler, AlphaEvolve
Bring-up / codesign	Coverage→perf on sim+silicon; ISA/IR feedback	TritorX, KernelEvolve, Ascend diagnosis, KForge

Classical AI compiler stack (substrate)

```
Framework capture     torch.compile / Dynamo, JAX, TF graphs
        ↓
Portable HLO          StableHLO (TF/JAX/PyTorch ↔ XLA/IREE)
        ↓
MLIR dialects         multi-level lowers, rewrites, vendor dialects
        ↓
Schedule / kernels    TVM MetaSchedule, Inductor→Triton, CUDA Tile IR, CUTLASS
        ↓
Serving runtime       vLLM, FlashAttention, CUDA Graphs, decode paths
        ↓
CPU / legacy IR       LLVM opt pipelines, PGO / AutoFDO, MLGO advisors
```

Canonical hybrid loop

```
Capture → Analyze regions → Agent proposes
        → Compiler checks & lowers
        → Verify / test (empirical or formal)
        → Benchmark / select
        → Feedback to orchestrator
        → Fallback if unprofitable
```

Invariant: LLM outputs guide search; they should not silently define unchecked executable behavior.

Publications index

Publications index

One digest per searched source. Files live beside this index.

Year	Kind	Group	Digest	Source
2026	paper	Surveys & vision	The New Compiler Stack: A Survey on the Synergy of LLMs and Compilers	source
2025	paper	Surveys & vision	Compiler.next: A Search-Based Compiler to Power the AI-Native Future of Software Engineering	source
2026	paper	Surveys & vision	Reading AI Model Compilation in MLIR Through the Lens of Formal Theories	source
2018	paper	Classic DL compilers	TVM: An Automated End-to-End Optimizing Compiler for Deep Learning	source
2020	paper	Classic DL compilers	Ansor: Generating High-Performance Tensor Programs for Deep Learning	source
2021	company	Classic DL compilers	TVM blog: Introducing Auto-scheduler (Ansor)	source
2020	paper	Classic DL compilers	FlexTensor: An Automatic Schedule Exploration and Optimization Framework	source
2022	paper	Classic DL compilers	TensorIR: An Abstraction for Automatic Tensorized Program Optimization	source
2021	paper	Classic DL compilers	MLIR: A Compiler Infrastructure for the End of Moore's Law	source
2025	company	Classic DL compilers		source

Year	Kind	Group	Digest	Source
			OpenXLA StableHLO roadmap	
2021	paper	MLGO & RL gyms	MLGO: a Machine Learning Guided Compiler Optimizations Framework	source
2022	company	MLGO & RL gyms	Google Research blog: MLGO	source
2022	company	MLGO & RL gyms	InfoQ: MLGO Framework Brings Machine Learning in Compiler Optimizations	source
2022+	code	MLGO & RL gyms	LLVM docs: Machine Learning Guided Optimization (MLGO)	source
2022	code	MLGO & RL gyms	CompilerGym (Meta/FAIR)	source
2022	forum	MLGO & RL gyms	LLVM Discourse: ML-guided ordering of compiler optimization passes (GSoC)	source
2023	paper	Foundation LLMs for compilers	Large Language Models for Compiler Optimization	source
2024	paper	Foundation LLMs for compilers	Meta Large Language Model Compiler: Foundation Models of Compiler Optimization	source
2024	company	Foundation LLMs for compilers	Meta AI publication page: LLM Compiler	source
2024	company	Foundation LLMs for compilers	Chris Cummins: LLM Compiler foundation models post	source

Year	Kind	Group	Digest	Source
2024	paper	Foundation LLMs for compilers	Compiler generated feedback for Large Language Models	source
2023	forum	Foundation LLMs for compilers	HN: Large Language Models for Compiler Optimization	source
2024	forum	Foundation LLMs for compilers	HN: Meta LLM Compiler (long thread)	source
2024	forum	Foundation LLMs for compilers	HN: Meta Large Language Model Compiler	source
2025	paper	Agentic & RL compilers	Compiler-R1: Towards Agentic Compiler Auto-tuning with Reinforcement Learning	source
2025	code	Agentic & RL compilers	Mind4Compiler/ Compiler-R1 (code)	source
2026	paper	Agentic & RL compilers	LLM-VeriOpt: Verification-Guided RL for LLM-Based Compiler Optimization	source
2026	paper	Agentic & RL compilers	Magellan: Autonomous Discovery of Novel Compiler Optimization Heuristics with AlphaEvolve ★ Tier A offline job	source
2025	talk	Agentic & RL compilers	LLVM Developers' Meeting slides: Magellan ★ future signals (§5.4)	source
2025	paper	Agentic & RL compilers	AlphaEvolve: A coding agent for scientific and algorithmic discovery	source
2026	company	Agentic & RL compilers		source

Year	Kind	Group	Digest	Source
			DeepMind blog: AlphaEvolve impact	
2025	paper	Agentic & RL compilers	AwareCompiler: Agentic Context-Aware Compiler Optimization	source
2026	paper	Agentic & RL compilers	AutoPass: Evidence-Guided LLM Agents for Compiler Performance Tuning	source
2026	paper	Agentic & RL compilers	HintPilot: LLM-based Compiler Hint Synthesis for Code Optimization	source
2026	paper	Agentic & RL compilers	Agentic Code Optimization via Compiler-LLM Cooperation (ACCLAIM) ★ Tier A Q2/Q3	source
2026	code	Agentic & RL compilers	amazon-science/acclaim (GitHub) ★ Tier A	source
2026	paper	Agentic & RL compilers	Generative Compilation: On-the-Fly Compiler Feedback as AI Generates Code	source
2025	paper	GPU kernels & inference compilers	KernelBench: Can LLMs Write Efficient GPU Kernels?	source
2025	company	GPU kernels & inference compilers	Stanford blog: KernelBench	source
2025	code	GPU kernels & inference compilers	ScalingIntelligence/KernelBench	source
2026	paper	GPU kernels & inference compilers	KernelBench-X: A Comprehensive Benchmark for Evaluating LLM-	source

Year	Kind	Group	Digest	Source
			Generated GPU Kernels	
2025	paper	GPU kernels & inference compilers	GEAK: Introducing Triton Kernel AI Agent & Evaluation Benchmarks	source
2025	company	GPU kernels & inference compilers	AMD ROCm blog: Triton kernel AI / GEAK	source
2025	code	GPU kernels & inference compilers	AMD-AGI/GEAK agent	source
2025	company	GPU kernels & inference compilers	Meta KernelLLM (Hugging Face)	source
2025	paper	GPU kernels & inference compilers	Reasoning Compiler: LLM-Guided Optimizations for Efficient Model Serving	source
2026	paper	GPU kernels & inference compilers	AgentCompile: An LLM-Guided Compiler for Direct CUDA Inference	source
2025	code	GPU kernels & inference compilers	ucb-bar/mlirAgent	source
2025	company	Company infra	NVIDIA: Focus on Your Algorithm—CUDA Tile Handles the Hardware	source
2026	company	Company infra	NVIDIA: Develop High-Performance GPU Kernels in C++ with CUDA Tile	source
2026	company	Company infra	NVIDIA CUDA 13.3: Tile C++ + CompileIQ	source
2026	company	Company infra	NVIDIA: Extract More Kernel Performance with CompileIQ	source
2026	code	Company infra	CompileIQ documentation	source

Year	Kind	Group	Digest	Source
2025	company	Company infra	Modular: What about the MLIR compiler infrastructure?	source
2026	company	Company infra	Anthropic: Building a C compiler with a team of parallel Claudes	source
2026	company	Company infra	Ars Technica: Sixteen Claude agents created a C compiler	source
2026	company	Company infra	Modular/Lattner: The Claude C Compiler — Future of Software	source
2026	forum	Company infra	HN: We tasked Opus 4.6 agent teams to build a C Compiler	source
2025	forum	Forums & workshops	LLVM Discourse: LLVM ♥ ML Workshop 2025 agenda	source
2026	forum	Forums & workshops	LLVM Discourse: LLVM ♥ ML Workshop 2026 CFP	source
2023	forum	Forums & workshops	LLVM Discourse: ML-Guided Compiler Optimization Workshop 2023	source
2026	forum	Forums & workshops	LLVM Discourse: Automated review with agents (~30 bugs / 207 PRs)	source
2026	forum	Forums & workshops	comp.compilers: Magellan paper notice	source
2025	company	Forums & workshops	IEEE Pulse: LLMs in Compiler Optimization — Challenges and Future Direction	source

Year	Kind	Group	Digest	Source
2026	company	Forums & workshops	Moonlight literature review: Magellan	source
2010s+	code	Correctness lineage	Souper: A superoptimizer for LLVM IR	source

2026 | paper | Source control & review agents | Archer: Towards Agentic Review for Compiler Optimizations | source |
2026 | code | Source control & review agents | cuhk-s3/Archer (GitHub) | source |
2024+ | code | Source control & review agents | Gerrit plugin: ai-code-review (googlesource) | source |
2025+ | code | Source control & review agents | amarula/reviewai-gerrit-plugin | source |
2025+ | code | Source control & review agents | GerritForge/ai-review-agent-provider | source |
2025 | code | Source control & review agents | OpenEvolve (AlphaEvolve-style OSS) | source |
2025 | code | Source control & review agents | HeuriGym (cornell-zhang/heurigym) | source |
2025 | code | Source control & review agents | meta-pytorch/KernelAgent | source |
2026 | code | Source control & review agents | NVIDIA/CompileIQ (GitHub) | source |
2026 | code | Source control & review agents | anthropics/claudes-c-compiler | source |
2021+ | code | Source control & review agents | google/ml-compiler-opt | source |
2026 | code | Source control & review agents | ZJU-PL/hintpilot | source |
ongoing | code | Source control & review agents | llvm/llvm-project (host repository) | source |
2026 | paper | Surveys & vision | Towards Automated Kernel Generation in the Era of LLMs | source |
2025+ | code | GPU kernels & inference compilers | flagos-ai/awesome-LLM-driven-kernel-generation | source |
2026 | company | Commercial products & proposals | AMD ROCm: GEAK v3 kernel optimization agent | source |
2025 | forum | Forums & workshops | LLVM Discourse RFC: EmitC support for MLGO | source |
2026 | code | Commercial products & proposals | TensorRT-LLM PR: Claude agents/skills for kernels and compile | source |
2026 | company | Commercial products & proposals | CompileIQ docs: expected gains (2 to 3 percent highly optimized) | source |
2025/26 | paper | HW codesign & accelerator bring-up | Agentic Operator Generation for ML ASICs (TritorX) ★ bring-up / codesign | source |
2025/26 | paper | HW codesign & accelerator bring-up | KernelEvolve: Agentic Kernel Coding for Heterogeneous Accelerators ★ multi-HW prod | source |
2026 | paper | HW codesign & accelerator bring-up | Compiler-Grounded Hierarchical Diagnosis for Triton on NPUs ★ Ascend | source |
2026 | paper | HW codesign & accelerator bring-up | KForge: Cross-Platform Kernel Generation for AI Accelerators ★ multi-DSL | source |
2026 | paper | GPU kernels & inference compilers | AutoKernel: Autonomous GPU Kernel Optimization ★ Amdahl agent loop | source |
2026 | code | GPU kernels & inference compilers | RightNow-AI/autokernel (GitHub) ★ | source |
2026 | paper | GPU kernels & inference compilers | Kernel Forge: Agent Harness for CUDA Kernel Opt | source |

2026 | paper | GPU kernels & inference compilers | KernelBlaster: Memory-Augmented In-Context RL for CUDA | source |
2025 | company | Classic DL compilers | Helion: High-Level DSL for Portable ML Kernels | source |
2025+ | code | Classic DL compilers | pytorch/helion (GitHub) | source |

Total: 95 digests

Kinds: `paper` · `company` · `forum` · `talk` · `code`

★ = high-signal for next-gen **prediction** (prefer these when updating `docs/SURVEY.md` §5 / `docs/ROADMAP.md`).

See also: `docs/REPOS.md` / `docs/PRODUCTS.md` as **Tier A/B/C evidence** (not catalogs); `docs/CONFLICTS.md` when sources disagree; `docs/STACK.md` for SW+HW reshape.

Export notes

- Full digests remain in `publications/*.md` (not inlined).
- Tier maps: `docs/REPOS.md`, `docs/PRODUCTS.md`.
- Rebuild: `python3 publish/build_pdf.py` (builds en / zh-CN / zh-TW).